

蓝桥杯30天算法冲刺集训



Day-22 有向图的连通性与 Tarjan

前置概念

给定有向图 $G = (V, E)$ ，若存在 $r \in V$ ，满足从 r 出发能够到达 V 中所有的点，则称 G 是一个“流图”(Flow Graph)，记为 (G, r) ，其中 r 称为流图的源点。

与无向图的深度优先遍历类似，我们也可以定义“流图”的搜索树和时间戳的概念：

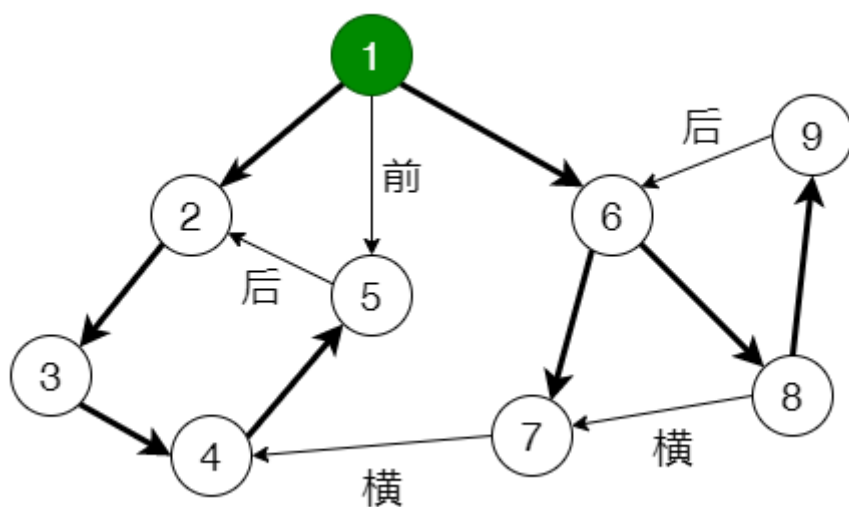
在一个流图 (G, r) 上从 r 出发进行深度优先遍历，每个点只访问一次。所有发生递归的边 (x, y) （换言之，从 x 到 y 是对 y 的第一次访问）构成一棵以 r 为根的树，我们把它称为流图 (G, r) 的**搜索树**。

同时，在深度优先遍历的过程中，按照每个节点第一次被访问的时间顺序，依次给予流图中 N 节点 $1 \sim N$ 的整数标记，该标记被称为**时间戳**，记为 $dfn[x]$ 。容易知道：一个结点的子树内结点的 dfn 都大于该结点的 dfn ；

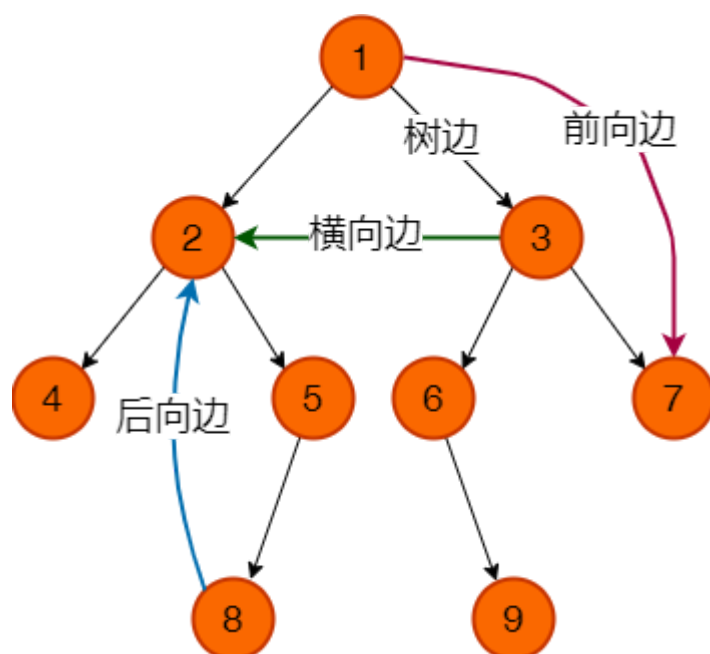
流图中的每条有向边 (x, y) 必然是以下四种之一（不一定全部出现）：

1. **树枝边** (tree edge)，指搜索树中的边，即 x 是 y 的父节点。
2. **前向边** (back edge)，指搜索树中 x 是 y 的祖先节点。
3. **后向边** (cross edge)，指搜索树中 y 是 x 的祖先节点。
4. **横叉边** (forward edge)，指除了以上三种情况之外的边，它一定满足 $dfn[y] < dfn[x]$ (y 在 x 前已经被搜索过了，且不是 x 的祖先节点)。

下图画出了一个“流图”以及它的搜索树、时间戳、边的分类。圆圈中的数字是时间戳。粗边是树枝边，并构成一棵搜索树。前向边、后向边与横叉边用第一个汉字标注。



另外一个例子：



我们考虑 搜索树（DFS 生成树）与强连通分量之间的关系。

如果结点 u 是某个强连通分量在搜索树中遇到的第一个结点，那么这个强连通分量的其余结点肯定是在搜索树中以 u 为根的子树中。结点 u 被称为这个强连通分量的根。

反证法：假设有个结点 v 在该强连通分量中但是不在以 u 为根的子树中，那么 u 到 v 的路径中肯定有一条离开子树的边。但是这样离开子树的边只可能是横叉边或者反祖边，然而这两条边都要求指向的结点已经被访问过了，这就和 u 是第一个访问的结点矛盾了。得证。

有向图的强连通分量

Tarjan 算法的核心原理

给定一张有向图。若对于图中任意两个节点 x, y 之间都连通，也就是既存在从 x 到 y 的路径，也存在从 y 到 x 的路径，则称该有向图是“强连通图”。

有向图的极大强连通子图被称为“强连通分量”，简记为 SCC (Strongly Connected Components)，也就是再增加任意一条图中的边都无法构成强连通子图。此处“极大”的含义与无向图中的双连通分量“极大”的含义类似。

Tarjan 算法基于有向图的深度优先遍历，能够在 **线性时间内** 求出一张有向图的各个强连通分量。

一个“环”一定是强连通图。如果既存在从 x 到 y 的路径，也存在从 y 到 x 的路径，那么 x, y 显然在一个环中。因此，**Tarjan 算法的基本思路就是对于每个点，尽量找到与它一起能构成环的所有节点。**

容易发现，“前向边” (x, y) 没有什么用处，因为搜索树上本来就存在从 x 到 y 的路径。“后向边” (x, y) 非常有用，因为它可以和搜索树上从 y 到 x 的路径一起构成环。“横叉边” (x, y) 视情况而定，如果从 y 出发能找到一条路径回到 x 的祖先节点，那么 (x, y) 就是有用的。

为了找到通过“后向边”和“横叉边”构成的环，Tarjan 算法在深度优先遍历的同时维护了一个栈。当访问到节点 x 时，栈中需要保存以下两类节点：

1. 搜索树上 x 的祖先节点，记为集合 $anc(x)$ 。
设 $y \in anc(x)$ 。若存在后向边 (x, y) ，则 (x, y) 与 y 到 x 的路径一起形成环。
2. 已经访问过，并且存在一条路径到达 $anc(x)$ 的节点。
设 z 是一个这样的点，从 z 出发存在一条路径到达 $y \in anc(x)$ 。若存在横叉边 (x, z) ，则 (x, z) 、 z 到 y 的路径、 y 到 x 的路径形成一个环。

综上所述，栈中的节点就是能与从 x 出发的“后向边”和“横叉边”形成环的节点。进而可以引入“追溯值”的概念。

追溯值

设 $subtree(x)$ 表示流图的搜索树中以 x 为根的子树。 x 的追溯值 $low[x]$ 定义为 x 或 x 的子树能够回溯到的最早的栈中节点的 dfn 值。也就是满足以下条件的节点的最小时间戳：

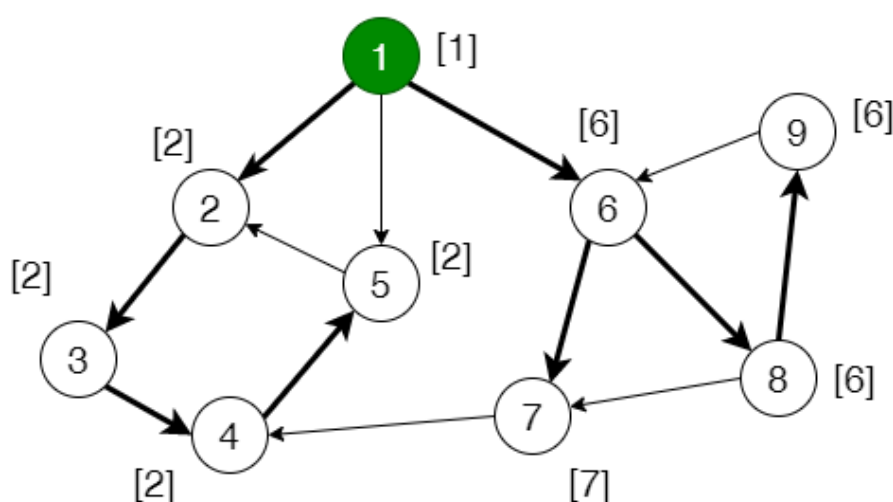
1. 该点在栈中。
2. 存在一条从 $subtree(x)$ 出发的有向边，以该点为终点。

容易知道，从根开始的一条路径上的 dfn 严格递增， low 严格非降；

根据定义，Tarjan 算法按照以下步骤计算“追溯值”：

1. 当节点 x 第一次被访问时，把 x 入栈，初始化 $low[x] = dfn[x]$ 。
2. 扫描从 x 出发的每条边 (x, y) 。
 - (1) 若 y 没被访问过，则说明 (x, y) 是树枝边， x 是 y 的父节点，递归访问 y ，从 y 回溯之后，令 $low[x] = \min(low[x], low[y])$ 。
 - (2) 若 y 被访问过并且 y 在栈中，则说明 (x, y) 是后向边或指向栈中节点的横叉边，令 $low[x] = \min(low[x], dfn[y])$ 。
3. 从 x 回溯之前，判断是否有 $low[x] = dfn[x]$ 。若成立，则不断从栈中弹出节点，直至 x 出栈。

下图中的中括号 $[]$ 里的数值标注了每个节点的“追溯值” low 。读者可以尝试在图中模拟 low 的计算过程。



强连通分量判定法则

在追溯值的计算过程中，若从 x 回溯前，有 $low[x] = dfn[x]$ 成立，则栈中 x 和到栈顶的所有节点构成一个强连通分量。

当 $dfn[u] = low[u]$ 时，以 u 为根的搜索子树上所有结点构成一个强连通分量。

简要证明：

大致来说，在计算追溯值的第 3 步，如果 $low[x] = dfn[x]$ ，那么说明 $subtree(x)$ 中的节点不能与栈中其他节点一起构成环。另外，因为横叉边的终点时间戳必定小于起点时间戳，所以 $subtree(x)$ 中的节点也不可能直接到达尚未访问的节点（时间戳更大）。综上所述，栈中从 x 到栈顶的所有节点不能与其他节点一起构成环。

又因为我们及时进行了判定和出栈操作，所以从 x 到栈顶的所有节点独立构成一个强连通分量。

详细证明：

在任何深度优先搜索中，同一强连通分量内的所有顶点均在同一棵深度优先搜索树中。也就是说，强连通分量一定是有向图的某一棵深度优先搜索树的子树的子集。

可以证明，当一个点既是强连通子图 1 中的点，又是强连通子图 2 中的点时，它是强连通子图 $1 \cup 2$ 中的点。

这样，我们用 low 值记录该点所在强连通子图对应的搜索子树的根结点的 dfn 值。注意，该子树中的元素在栈中一定是相邻的（因为我们及时进行了判定和出栈操作），且根结点在栈中一定位于所有子树元素的最下方。

强连通分量是由若干个环组成的。所以，当有环形成时（也就是搜索的下一个点已在栈中），我们将这一条路径的 low 值统一，即这条路径上的点属于同一个强连通分量（合并了两个强连通子图）。

如果遍历完整棵搜索树后，某个点的 dfn 值等于 low 值，则它是该搜索子树的根。这时，它以上（包括它自己）一直到栈顶的所有元素组成一个强连通分量。

参考代码：

下面的程序实现了 Tarjan 算法，求出数组 c ，其中 $c[x]$ 表示 x 所在的强连通分量的编号。另外，它还求出了 vector 数组 scc ， $scc[i]$ 记录了编号为 i 的强连通分量中的所有节点。整张图共有 cnt 个强连通分量。

```
#include<bits/stdc++.h>
using namespace std;
const int N = 1e5+5, M = 1e6+5;
int n, m, num, top, cnt; // num: 强连通分量编号
// ins[i]: i 是否在栈中 c[i]: i 所属强连通分量编号
int stk[N], ins[N], c[N];
int dfn[N], low[N];
vector<int> scc[N];
vector<int> g[N];

void tarjan(int u) {
    dfn[u] = low[u] = ++num;
    stk[++top] = u; // 首次访问，入栈
    ins[u] = 1; // 标记在栈中
    for (auto v : g[u]) { // 枚举每条出边
        if (!dfn[v]) { // 节点 v 未被访问过
            tarjan(v); // 继续向下找
            low[u] = min(low[u], low[v]); // (u,v) 是树枝边
        }
        else if (ins[v]) { // (u,v) 是后向边或横叉边， 结点 v 还在栈内，即 v 不属于任何强连通分量
            low[u] = min(low[u], dfn[v]);
        }
    }
    if (dfn[u] == low[u]) { // 结点 u 是强连通分量的根
        cnt++; int v;
        do {
            v = stk[top--], ins[v] = 0; // 将 v 出栈，为该强连通分量中一个顶点
            c[v] = cnt;
            scc[cnt].push_back(v);
        } while (u != v);
    }
}
```

```

}
int main() {
    cin >> n >> m;
    for(int i = 1; i <= m; i++) {
        int x, y;
        cin >> x >> y;
        g[x].push_back(y);
    }
    for(int i = 1; i <= n; i++) {
        if(!dfn[i])
            tarjan(i);
    }
    cout << cnt << endl;
    for(int i = 1; i <= cnt; i++) {
        cout << i << ":";
        for(auto u : scc[i]) {
            cout << u << ' ';
        }
        cout << endl;
    }
    //
}

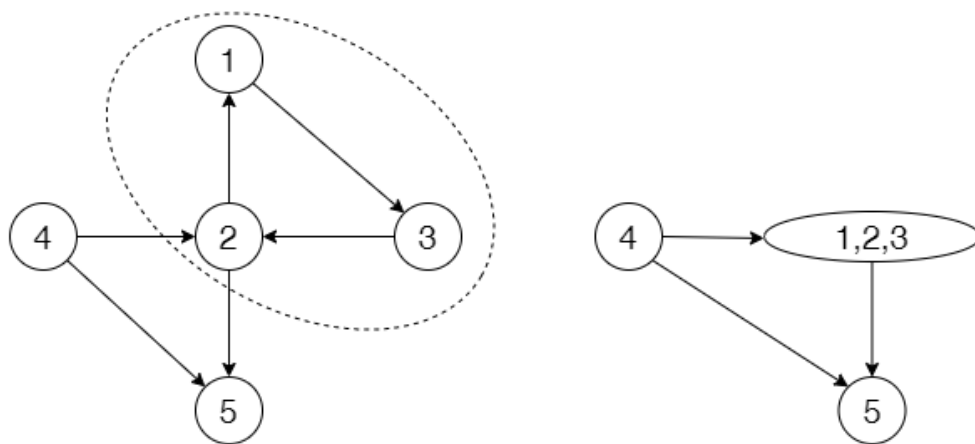
```

应用

有向图的缩点

我们可以将一张图的每个强连通分量都缩成一个点。

将同一个强连通分量中的点缩成一个新的节点，对于两个新节点 a, b ，它们之间有边相连，当且仅当存在两个点 u 属于 a ， v 属于 b ，且 $\langle u, v \rangle \in E$ 。例如下图。



经过缩点之后的图会变成一个有向无环图（DAG），这样就可以进行拓扑排序以及更多其他操作。

举个简单的例子，求一条路径，可以经过重复结点，要求经过的不同结点数量最多。

下面代码对 SCC 执行缩点过程，并把新得到的有向无环图保存到另一个邻接表中。

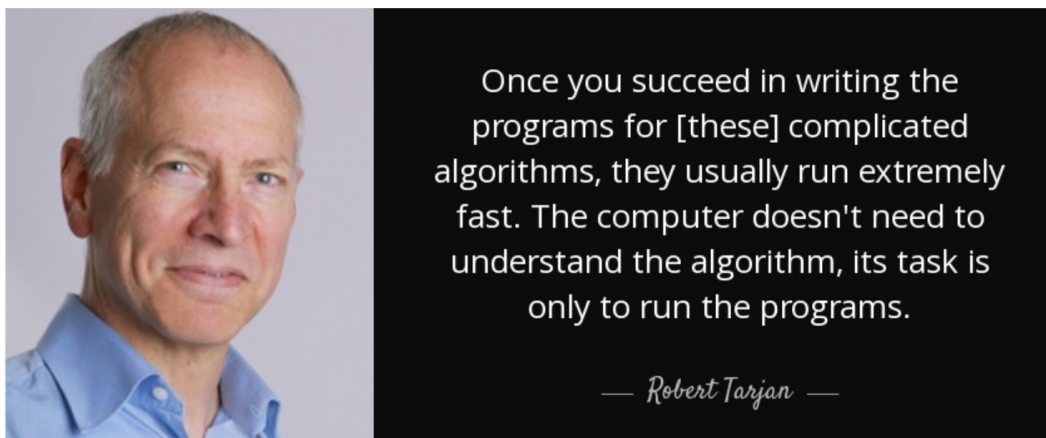
```
vector<int> gn[N];
// 在 main 中
for(int u = 1; u <= n; u++) {
    for(auto v : g[u]) {
        if(c[u] != c[v])    // 不在同一强连通分量中，边不会被缩点
            gn[c[u]].push_back(c[v]);
    }
}
```

蓝桥杯真题

Note

蓝桥杯省赛国赛直接使用有向图Tarjan的题目并不多，有的是在LCA上面再使用Tarjan，我觉得这样为了做题而做题并不可取，毕竟那些题目就是朴素显眼的LCA。于是这里就不放置真题了，大家有兴趣可以做一做习题加深一下印象，目前有向图的Tarjan考法还是挺固定的，基本就是用在缩点上面。

下面是Tarjan老爷子的图片。



习题

学校网络

先缩点，然后建图。

对于第一问，答案就是缩点后的图上入度为 0 的点的数量。原因如下：

1. 首先，缩点后的图上没有环，是有向无环图；
2. 其次，入度为 0 的学校不能由其他学校支援软件，因此对于每个入度为 0 的点，必然要直接给它提供软件；
3. 最后，对于入度不为 0 的点，追寻它的上游结点，必然最终能找到一个入度为 0 的点，因此，不需要为其直接提供软件。

综上，得证。

对于第二答案是入度为 0 的点的数量（记为 P ）和出度为 0 的点的数量（记为 Q ）中的最大值。原因如下（感性证明）：

1. 首先，如果只想给一个学校提供软件，其他学校不提供，那么加完边后的图必然是一个 SCC；
2. 那么，我们至少要保证所有入度为 0 的点各自至少要加 1 条**入边**，总共至少 P 条，所有出度为 0 的点各自至少要加 1 条**出边**，总共至少 Q 条；

3. 而上面这两种边的方向是不同的，一种是入边，一种是出边。为了保证加的边最少，我们可以对其两两组合出 $\min(P, Q)$ 条边，剩下的边再任意组合即可，最终至少要加 $P + Q - \min(P, Q) = \max(P, Q)$ 条边。

需要注意的边界情况：如果图是一个 SCC，那么就不需要加任何边，答案就是 0。

问题一

结论

只要把软件给所有起点即可，答案为起点个数 src 。

证明

所有起点都无法由别的点到达，因此每个起点必须分配一个软件，而对于其他所有点，一定存在前驱，一定能由某一个起点走到，也就是说从所有起点出发，能遍历整个图。因此只需要给所有起点各一个软件即可。

问题二

结论

1. 若 $scc_cnt = 1$ （只有一个强连通分量），则不需要连新的边，答案为 0。
2. 若 $scc_cnt > 1$ ，则答案为 $\max(src, des)$ 。

证明

结论 1 正确性显然，下面证明结论 2。

设缩点后的 DAG 中，起点（入度为 0）的集合为 P ，终点（出度为 0）的集合为 Q 。分以下两种情况讨论：

1. $|P| \leq |Q|$
 - ① 若 $|P| = 1$ ，则只有一个起点，并且这个起点能走到所有点，只要将每一个终点都向这个起点连一条边，那么对于图中任意一点，都可以到达所有点，新加的边数为 $|Q|$ 。
 - ② 若 $|P| \geq 2$ ，则 $|Q| \geq |P| \geq 2$ ，此时至少存在 2 个起点 p_1, p_2 ，2

个终点 q_1, q_2 ，满足 p_1 能走到 q_1 ， p_2 能走到 q_2 。（反证法：如果不存在两个起点能走到不同的终点，则所有的起点一定只能走到同一个终点，而终点至少有两个，发生矛盾，假设不成立）。如下图：

那么我们可以从 q_1 向 p_2 新连一条边，那么此时起点和终点的个数都会减少一个（ p_2 不再是起点， q_1 不再是终点），因此只要以这种方式，连接新边 $|P| - 1$ 条，则 $|P'| = 1$ ，而 $|Q'| = |Q| - (|P| - 1)$ ，由 ① 得，当 $|P'| = 1$ 时，需要再连 $|Q'|$ 条新边，那么总添加的新边数量为 $|P| - 1 + |Q| - (|P| - 1) = |Q|$ 。

$$1. |Q| \leq |P|$$

与情况 1 对称，此时答案为 $|P|$ 。

综上所述， $scc_cnt > 1$ 时，问题二的答案为 $\max(|P|, |Q|)$ 。

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5+5;
int n, m, scc_cnt;
vector<int> g[N];
int dfn[N], low[N], timestamp;
int stk[N], top, in_stk[N], id[N];
vector<int> scc[N];
// 统计新图
vector<int> gn[N];
int din[N], dout[N]; // 缩点后各点入度和出度

void tarjan(int u) {
    dfn[u] = low[u] = ++timestamp;
    stk[++top] = u;
    in_stk[u] = 1;
    for(auto v : g[u]) {
        if(!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        } else if(in_stk[v]) {
            low[u] = min(low[u], dfn[v]);
        }
    }
}
```

```

    if(dfn[u] == low[u]) {
        ++scc_cnt; int v;
        do {
            v = stk[top--], in_stk[v] = 0;
            id[v] = scc_cnt;
            scc[scc_cnt].push_back(v);
        } while(u != v);
    }
}

int main(){
    cin >> n;
    for(int i = 1; i <= n; i++) {
        int t;
        while(cin >> t && t) {
            g[i].push_back(t);
        }
    }
    for(int i = 1; i <= n; i++) {
        if(!dfn[i]) {
            tarjan(i);
        }
    }

    for(int u = 1; u <= n; u++) {
        for(auto v : g[u]) {
            int iu = id[u], iv = id[v];
            if(iu != iv) {
                dout[iu]++;
                din[iv]++;
                //gn[c[u]].push_back(c[v]);
            }
        }
    }

    int sin = 0, sout = 0; // 记入度和出度为 0 的点的数量
    for(int i = 1; i <= scc_cnt; i++) {
        if(!din[i]) sin++;
        if(!dout[i]) sout++;
    }
}

```

```
cout << sin << '\n';  
if(scc_cnt == 1) // 如果全强连通，不用加新边  
    cout << 0;  
else  
    cout << max(sin, sout);  
return 0;  
}
```